CONFIDENTIAL PATENT DISCLOSURE DOCUMENT — VERSION 2.0

HealthLedger AI

Accounting Intelligence for Healthcare

AI-powered compliance built specifically for hospital accounting departments. Custom-trained on cooperation agreements. Grounded in doctoral research. Built from the inside — through on-site immersion.

Inventor:	Maria Polychroniadou
Contact:	polychroniadou@gmx.de
Date of Disclosure:	June 26, 2026
Status:	Confidential Pre-Filing Disclosure
Document Version:	2.0 — Full Technical & Code Disclosure
Prepared for:	Patent Counsel — Attorney Review

TABLE OF CONTENTS

1. FIELD OF THE INVENTION

The present invention relates to artificial intelligence systems applied to healthcare financial administration. More particularly, the invention concerns a managed AI agent system trained on hospital-specific cooperation agreement corpora and deployed as a real-time, clause-traceable compliance and decision-support tool for hospital accounting departments. The system further encompasses a novel on-site immersion methodology for capturing institution-specific accounting knowledge as training data.

2. BACKGROUND OF THE INVENTION

Hospital accounting departments operate within a uniquely complex regulatory and contractual environment. The following problems, identified through doctoral research and direct on-site immersion with hospital finance teams, motivate the present invention:

■ **Volume and Complexity of Cooperation Agreements.** A single hospital network may maintain in excess of 200 active cooperation agreements with insurers, pharmaceutical suppliers, medical device manufacturers, rehabilitation providers, and other healthcare entities. Each agreement contains clauses carrying direct accounting obligations including revenue recognition triggers, cost allocation rules, performance thresholds, and compliance deadlines. The total clause volume across a hospital network can exceed several thousand individually actionable accounting obligations.

■ **Inconsistent Clause Interpretation.** In the absence of a machine-readable authoritative source, cooperation agreement clauses are interpreted inconsistently across accounting staff, teams, fiscal periods, and organisational units. The same clause may yield materially different booking decisions depending on the individual accountant consulted. No prior art system addresses this structural inconsistency in healthcare contract accounting.

■ **Material Financial Exposure.** A single misread or inconsistently applied clause in a cooperation agreement can generate liability ranging from tens of thousands to hundreds of thousands of euros per fiscal period. Existing tools — general-purpose ERP systems, document management platforms, and generic large language model assistants — are not trained on, and are incapable of reasoning correctly about, the specific contractual language of a given institution's cooperation agreements.

■ **Absence of Domain-Specific AI.** No commercially available AI system has been developed specifically for hospital cooperation agreement compliance. General-purpose AI assistants lack: (a) training on healthcare cooperation agreement corpora; (b) knowledge of institution-specific charts of accounts and accounting conventions; and (c) the clause-traceable response architecture required for auditability in regulated financial environments.

■ **Tacit Knowledge Gap.** Hospital accounting teams accumulate substantial tacit interpretive knowledge — informal conventions, precedent decisions, and team-specific practices — that is not

captured in any written document and therefore not accessible to any prior art automated system. The present invention addresses this gap through its on-site immersion methodology.

---

3. SUMMARY OF THE INVENTION

HealthLedger AI is an artificial intelligence compliance system comprising a managed AI agent architecture trained on hospital-specific cooperation agreements and deployed as a real-time, clause-traceable compliance interface for hospital accounting departments. The system is characterised by the following four inventive pillars:

Pillar I — Cooperation Agreement Mapping

An automated document ingestion and semantic segmentation pipeline that parses hospital cooperation agreements, classifies clauses by accounting obligation type (revenue recognition, cost allocation, compliance trigger, penalty clause, revenue threshold), generates vector embeddings, and indexes the clause corpus in a queryable vector database. The pipeline enables semantic retrieval of clauses most relevant to a given accounting query.

Pillar II — Accounting Compliance Intelligence

A retrieval-augmented generation (RAG) layer that, upon receipt of an accounting query, retrieves the semantically closest agreement clauses, injects them into the AI agent's context, and generates a compliance determination that explicitly cites the source clause identifier and parent agreement. The determination is therefore fully auditable and defensible under accounting standards.

Pillar III — Hospital-Specific AI Model

A domain-adapted large language model agent configured with institution-specific instructions comprising: the hospital's chart of accounts; internal naming convention mappings; and historical interpretive decisions captured during on-site immersion. The agent is created as a persistent entity with a stable identity; individual user interactions are mediated through discrete, stateful sessions. This architecture enables the model to reason in the specific accounting language of the target institution.

Pillar IV — On-Site Immersion Methodology

A novel system development methodology in which the inventors engage in direct, sustained collaboration with hospital accounting teams in their working environment. This engagement produces ground-truth training data — actual interpretive decisions, rationales, and accounting conventions — that cannot be derived from publicly available documents or synthetic data generation. The methodology constitutes an independent inventive contribution.

4. DETAILED TECHNICAL DESCRIPTION

4.1 System Architecture Overview

The system is implemented as a multi-layer software architecture deployed over a secure HTTPS API infrastructure. The principal layers are:

- **Layer 1 — Document Knowledge Base:** A vector database containing semantically embedded representations of all agreement clauses for the target institution, indexed by clause type, agreement ID, and effective date.
- **Layer 2 — AI Agent Kernel:** A persistent managed AI agent entity instantiated via the Anthropic Managed Agents API (model: claude-opus-4-8). The agent holds a stable identity (persistent agent ID) and carries institution-specific instructions encoding the hospital's accounting conventions.
- **Layer 3 — Session Orchestration:** A session layer that mediates each user interaction as a discrete, stateful session against the persistent agent. Each session is initialised with a retrieval-augmented prompt constructed from the user query and semantically retrieved agreement clauses.
- **Layer 4 — Compliance Output Module:** A response post-processor that extracts the compliance determination, source clause citations, and confidence indicators from the agent response, and returns a structured output to the accounting user interface.
- **Layer 5 — Frontend Interface:** A Next.js 14 web application providing the public-facing compliance query interface, research programme enrolment portal, and the on-site immersion data capture tool for institutional knowledge ingestion.

4.2 AI Agent Module

The AI Agent Module is the core computational engine of the system. It implements the 'create once, reuse always' pattern of managed AI agent architecture:

Agent Creation: On initial system deployment for a given institution, the system calls the Anthropic agents.create() API method, supplying: (a) a unique agent name identifying the institution; (b) the target model identifier (claude-opus-4-8); and (c) a domain-specific instruction set assembled from the institution's accounting profile. The resulting agent ID is stored durably and serves as the permanent identity of the compliance AI for that institution.

Session Execution: Each user accounting query instantiates a new session via the sessions.create() API call, referencing the stored agent ID. The session receives a retrieval-augmented prompt and returns a streamed response. Response streaming enables real-time output to the accounting interface, with tokens delivered via server-sent events as they are generated.

Statefulness Model: The persistent agent ID carries the institution's accumulated domain knowledge across all sessions. Individual sessions are ephemeral. This architecture separates long-term institutional knowledge (in the agent) from per-query context (in the session), enabling efficient multi-user deployment without context contamination between users.

4.3 Agreement Ingestion & RAG Pipeline

Upon onboarding a new hospital institution, all cooperation agreements are processed by the Agreement Ingestion Pipeline:

■ **Step A — Document Parsing:** PDF cooperation agreements are parsed into plain text using a structured document parsing library. Page layout, headers, and section numbering are preserved as metadata.

■ **Step B — Clause Segmentation:** The parsed text is processed by an NLP clause boundary detection model that identifies individual clauses based on structural and semantic cues (section numbers, transitional language, obligation-bearing verb patterns).

■ **Step C — Obligation Type Classification:** Each segmented clause is classified into one of: RevenueRecognition, CostAllocation, ComplianceTrigger, RevenueThreshold, or PenaltyClause. Classification is performed by a fine-tuned text classification model or by the primary LLM with a classification prompt.

■ **Step D — Vector Embedding:** Each classified clause is converted to a high-dimensional vector embedding using a text embedding model, capturing its semantic content in a form suitable for similarity search.

■ **Step E — Vector Database Indexing:** Embeddings are stored in a vector database (e.g., Pinecone, Weaviate, or pgvector) with metadata: clauseId, agreementId, clauseType, effectiveDate, and raw text. This index is queried at compliance-query time.

■ **Step F — RAG Query Execution:** Upon receipt of an accounting query, the query text is embedded and subjected to approximate nearest-neighbour search over the clause index. The top-K most semantically similar clauses are retrieved and injected into the agent session prompt as structured context.

4.4 Domain Training & Institutional Knowledge Module

The Domain Training Module operationalises the on-site immersion methodology by converting captured institutional knowledge into agent instruction content:

■ **Chart of Accounts Integration:** The hospital's chart of accounts is ingested and encoded into the agent's instruction set. The agent is instructed to use the institution's exact account codes and names when generating compliance determinations.

■ **Naming Convention Mapping:** Internal institutional terminology that differs from standard accounting nomenclature is captured as a key-value mapping and embedded in the agent instructions. This prevents the AI from using unfamiliar terminology that would confuse accounting staff.

■ **Historical Interpretation Records:** Accounting decisions captured during on-site immersion are stored as structured records (clause reference, decision, rationale, date). The most recent N records are injected into the agent instructions as precedent examples, enabling few-shot learning from actual institutional decisions.

■ **Incremental Instruction Update:** As new interpretive decisions are captured post-deployment, the agent's instruction set is updated. Because instructions are stored at the agent level (not the

session level), updates take effect immediately for all subsequent sessions without retraining.

4.5 Live Compliance Engine

The Live Compliance Engine delivers real-time accounting guidance at the point of decision. The query lifecycle is as follows:

■ **Query Receipt:** An accounting staff member submits a natural-language compliance query via the web interface (e.g., 'How do we recognise revenue from the MedTech service contract in Q3?').

■ **Semantic Clause Retrieval:** The query is embedded and the five most semantically similar cooperation agreement clauses are retrieved from the vector database, ranked by cosine similarity score.

■ **Augmented Prompt Assembly:** A structured prompt is assembled combining: (a) the retrieved clauses with their agreement references; (b) the user query; and (c) an instruction to cite the source clause in the response.

■ **Agent Session Execution:** A session is created against the persistent institutional agent. The augmented prompt is submitted. The agent generates a response that reasons over the retrieved clauses and delivers a compliance determination.

■ **Streaming Output:** The response is streamed token-by-token to the accounting interface, enabling real-time display as the AI reasons. Target latency for first token: under 800ms. Full response: under 5 seconds.

■ **Structured Response Extraction:** The response is post-processed to extract: the compliance determination; cited clause identifiers; parent agreement references; and a confidence indicator. These are stored in the audit log alongside the original query.

4.6 Frontend Interface & Research Enrollment

The HealthLedger AI frontend is implemented as a Next.js 14 web application using React 18 with client-side interactivity. Key interface components are:

■ **ProblemHook Component:** Sequentially displays the three-part problem statement ('A hospital has 200+ cooperation agreements.', 'Each interpreted differently by accounting.', 'One clause. Thousands in exposure.') at 1,900ms intervals using framer-motion AnimatePresence. This constitutes the primary user engagement hook and contextualises the system's purpose.

■ **HeroScene Component:** Implements a mouse-reactive 3D parallax tilt effect using spring physics (framer-motion useSpring / useTransform), rotating the hero canvas on mouse movement. Provides a distinctive and memorable interface that differentiates the system visually.

■ **SectionHow Component:** Renders the three-step inventive method (Agreement Ingestion, Domain Training, Live Compliance) with scroll-triggered animation (useInView). Serves both as product explanation and as the primary product specification visible to prospective hospital accounting partners.

■ **SectionContact Component:** Collects hospital finance contact details for the on-site immersion research programme. Submitted contacts enter the on-site engagement pipeline, ultimately contributing to the Domain Training module's institutional knowledge base.

■ **Marquee Component:** Continuously scrolling ticker communicating system capabilities, providing a secondary messaging layer.

4.7 Data Flow & API Integration

The following table summarises the principal data flows within the system:

Data Flow	Source	Destination	Protocol
Agreement PDFs	Hospital IT/Legal	Ingestion Pipeline	HTTPS upload
Clause Embeddings	Embedding Model	Vector Database	API call
Accounting Query	Accountant UI	RAG Engine	HTTPS POST
Augmented Prompt	RAG Engine	Agent Session	Anthropic API
Streamed Response	Agent Session	Accountant UI	SSE stream
Audit Log Entry	Output Module	Compliance DB	DB write
On-Site Capture Record	Immersion Tool	Institution Profile	DB write
Updated Instructions	Domain Trainer	Agent (via API)	Anthropic API

4.8 On-Site Immersion Methodology

The On-Site Immersion Methodology is a novel approach to domain-specific AI development in regulated healthcare environments. It comprises three phases:

■ **Phase 1 — Embedded Engagement:** Inventors and/or system developers are physically present in the hospital accounting department for a sustained period (typically 4–12 weeks). During this phase, all accounting decisions involving cooperation agreements are observed, documented, and captured in the structured interpretation record format.

■ **Phase 2 — Knowledge Elicitation:** Structured interviews with accounting staff elicit tacit interpretive knowledge — conventions that are followed consistently but never written down. This knowledge is formalised into the naming convention map and the institutional instruction set.

■ **Phase 3 — Validation Loop:** Interim system outputs are reviewed by accounting staff for accuracy and appropriateness. Feedback is used to refine the agent instruction set and the clause segmentation pipeline prior to production deployment. This iterative validation is performed on-site, enabling rapid correction cycles unavailable in remote development contexts.

5. SOURCE CODE LISTING

The following source code listings constitute a full technical disclosure of the implementation. Listings include the actual production code from the repository and annotated pseudocode for inventive modules under development. All code is the original work of the inventor.

5.1 Core AI Agent Module

The following is the complete, verbatim source code of the core AI agent orchestration module (managed-agents-starter.ts). This module implements the fundamental create-once / reuse-always pattern of the managed agent architecture, constituting the technical heart of the invention:

Listing 5.1 — managed-agents-starter.ts (production code)

```
// FILE: managed-agents-starter.ts
// HealthLedger AI — Core Agent Orchestration Module
// Implements: persistent agent identity + session-based interaction pattern

import Anthropic from "@anthropic-ai/sdk";

const client = new Anthropic(); // Reads ANTHROPIC_API_KEY from environment

// ■■■ Agent Creation (run once per deployment) ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
// Creates a named, persistent agent entity with domain-specific instructions.
// The agent ID is stored durably (env / DB) and reused for every user session.

async function createAgent() {
const agent = await client.beta.agents.create({
name: "healthledger-assistant",
model: "claude-opus-4-8", // Large-scale transformer model
instructions:
"You are a compliance AI for HealthLedger. " +
"Help hospital accounting teams interpret cooperation agreements, " +
"identify accounting obligations, and apply correct booking rules. " +
"Always cite the exact clause and agreement you are referencing.",
});

console.log("Agent created:", agent.id);
// Store agent.id persistently — referenced in every subsequent session
console.log(`AGENT_ID=${agent.id}`);
return agent;
}

// ■■■ Session Execution (called per user interaction) ■■■■■■■■■■■■■■■■■■■■■■■■■
// Each accounting query opens a discrete session against the persistent agent.
// Sessions are stateful within a single interaction; agent state persists
// across all sessions via the stored agent identity.

async function runSession(agentId: string, userMessage: string) {
const session = await client.beta.sessions.create({
agent_id: agentId, // Links session to the persistent agent
});

console.log(`Session ${session.id} started`);

// Stream response events — enables real-time output to accounting UI
const stream = client.beta.sessions.events.stream(session.id, {
input: [{ role: "user", content: userMessage }],
});
```

```
process.stdout.write("HealthLedger AI: ");
for await (const event of stream) {
if (
event.type === "content_block_delta" &&
event.delta.type === "text_delta"
) {
process.stdout.write(event.delta.text); // Real-time streaming output
}
}
return session;
}

// ■■■ Entry Point ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async function main() {
const agentId = process.env.AGENT_ID; // Load stored agent ID

if (!agentId) {
// First-time setup: create the agent, store its ID
const agent = await createAgent();
console.log(`Re-run with: AGENT_ID=${agent.id} npx ts-node ...`);
return;
}

// Production: reuse existing agent for every accounting compliance query
await runSession(agentId,
"Under Agreement #47, section 3.2, how should we recognize " +
"the Q3 device maintenance revenue from MedTech GmbH?"
);
}

main().catch(console.error);
```

5.2 Agreement Ingestion & RAG Pipeline

The following pseudocode details the Agreement Ingestion and Retrieval-Augmented Generation pipeline. This constitutes the technical implementation of Pillar I (Cooperation Agreement Mapping) and the retrieval mechanism underlying Pillar II (Accounting Compliance Intelligence):

Listing 5.2 — Agreement Ingestion & RAG (annotated pseudocode)

```
// PSEUDOCODE: Agreement Ingestion & RAG Pipeline
// FILE: lib/ingestion-pipeline.ts (inventive module)

interface AgreementClause {
clauseId: string; // Unique clause identifier
agreementId: string; // Parent agreement reference
text: string; // Raw clause text
type: ClauseType; // "revenue" | "cost" | "compliance" | "threshold"
obligations: string[]; // Extracted accounting obligations
effectiveDate: Date;
embedding?: number[]; // Vector embedding for semantic search
}

enum ClauseType {
RevenueRecognition = "revenue",
CostAllocation = "cost",
ComplianceTrigger = "compliance",
RevenueThreshold = "threshold",
PenaltyClause = "penalty",
```

```
}

// Step 1: Document Ingestion
async function ingestAgreement(pdfPath: string, agreementId: string) {
const rawText = await parsePDF(pdfPath); // PDF → plain text
const clauses = await segmentClauses(rawText); // NLP clause boundary detection
const tagged = await tagClauseTypes(clauses); // Classify each clause by type
const embeddings = await embedClauses(tagged); // Generate vector embeddings
await storeInVectorDB(embeddings, agreementId); // Index for retrieval
return embeddings;
}

// Step 2: Retrieval-Augmented Generation query
async function queryCompliance(
agentId: string,
userQuery: string
): Promise<ComplianceResponse> {

// 2a. Semantic search over the vector database
const queryEmbedding = await embedQuery(userQuery);
const topClauses = await vectorDB.search(queryEmbedding, { topK: 5 });

// 2b. Assemble context: inject retrieved clauses into agent session
const context = topClauses.map(c =>
`[${c.agreementId} § ${c.clauseId}]: ${c.text}`
).join("\n\n");

const augmentedPrompt = `
Context clauses from cooperation agreements:
${context}

Accounting question: ${userQuery}

Provide a compliance determination citing the exact clause above.
`;

// 2c. Run against the persistent HealthLedger agent
const session = await client.beta.sessions.create({ agent_id: agentId });
const response = await streamSession(session.id, augmentedPrompt);

return {
determination: response.text,
sourceClauses: topClauses.map(c => c.clauseId),
agreementRefs: topClauses.map(c => c.agreementId),
confidence: response.confidence,
timestamp: new Date(),
};
}
```

5.3 Domain Training Module

The following pseudocode details the Domain Training and Institutional Knowledge Capture module. This implements Pillar III (Hospital-Specific AI) and operationalises Pillar IV (On-Site Methodology) by formalising captured knowledge into agent instructions:

Listing 5.3 — Domain Training Module (annotated pseudocode)

```
// PSEUDOCODE: Domain Training & Institutional Knowledge Capture
// FILE: lib/domain-training.ts (inventive module)
```

```
interface InstitutionProfile {
hospitalId: string;
chartOfAccounts: AccountCode[]; // Institution's own account numbering
namingConventions: Record<string, string>; // Internal terminology map
interpretations: InterpretationRecord[]; // Captured on-site decisions
}

interface InterpretationRecord {
agreementClauseRef: string; // Which clause was interpreted
decision: string; // What booking decision was made
rationale: string; // Expert justification (on-site captured)
capturedBy: string; // Finance team member
capturedAt: Date;
}

// On-Site Immersion Data Capture
// Accounting team members log their interpretive decisions into the system
// during the on-site engagement period. This produces ground-truth training data.
async function captureInterpretation(record: InterpretationRecord) {
await db.interpretations.insert(record);
await retrainAgentInstructions(record); // Incrementally update agent context
}

// Assemble institution-specific instruction set for agent domain training
async function buildInstitutionInstructions(
hospitalId: string
): Promise<string> {
const profile = await db.institutions.findById(hospitalId);
const history = await db.interpretations.findByHospital(hospitalId);

return `
You are the HealthLedger AI compliance assistant for ${profile.name}.

Chart of Accounts (use these exact codes):
${profile.chartOfAccounts.map(a => ` ${a.code}: ${a.name}`).join("\n")}

Internal naming conventions:
${Object.entries(profile.namingConventions)
.map(([k, v]) => ` "${k}" refers to "${v}"`)
.join("\n")}

Historical interpretive decisions (learned from on-site immersion):
${history.slice(-20).map(h =>
` Clause ${h.agreementClauseRef}: "${h.decision}" — ${h.rationale}`
).join("\n")}

Always cite the source clause. Always use the institution's account codes.
Acknowledge uncertainty when a clause is ambiguous.
`;
}
```

5.4 Frontend Application — app/page.tsx (Excerpts)

The following excerpts from the Next.js frontend application demonstrate the data structures and component constants that directly reflect the inventive system architecture — including the problem statement hooks, the four pillars, and the three-step method as presented to hospital accounting users:

Listing 5.4 — app/page.tsx (production code, excerpts)

```
// FILE: app/page.tsx (Next.js 14 — React Server + Client Components)
// HealthLedger AI — Public-facing interface & research enrollment

"use client";

// Problem framing — displayed before the main interface loads
const PROBLEM_HOOKS = [
"A hospital has 200+ cooperation agreements.",
"Each interpreted differently by accounting.",
"One clause. Thousands in exposure.",
];

// Four core value pillars — correspond directly to the four patent claims
const PILLARS = [
"Cooperation Agreement Mapping",
"Accounting Compliance",
"Hospital-Specific AI",
"On-Site Methodology",
];

// Three-step method — maps to Steps 01–03 of the inventive process
const STEPS = [
{
num: "01", title: "Agreement Ingestion",
body: "Map every clause in cooperation agreements — identifying " +
"accounting obligations, revenue thresholds, and compliance " +
"triggers specific to your hospital network.",
},
{
num: "02", title: "Domain Training",
body: "The AI learns the accounting language of your institution: " +
"your chart of accounts, internal naming conventions, and the " +
"interpretations your finance team relies on.",
},
{
num: "03", title: "Live Compliance",
body: "Real-time guidance at the point of decision. When an accounting " +
"question touches a cooperation agreement, the answer is immediate " +
"— and traceable to the source clause.",
},
];

// Research enrollment — hospital accounting teams apply for on-site program
const handleResearchEnrollment = (email: string) => {
window.location.href =
"mailto:info@healthledgerai.com" +
"?subject=Research%20Program%20Interest" +
"&body=Hello%2C%20I%27d%20like%20to%20learn%20more%20about%20HealthLedger%20AI.";
};
```

5.5 UI Component Architecture

The following code excerpts detail the key React components implementing the user interface layer of the system:

Listing 5.5 — Component architecture (production code, excerpts)

```
// FILE: components/HeroScene.tsx — 3D Parallax Scene Wrapper
// Implements mouse-reactive 3D tilt effect on the hero canvas.
// Demonstrates the interactive UX layer of the compliance interface.

import { motion, useMotionValue, useSpring, useTransform } from "framer-motion";

export default function HeroScene({ children }) {
const rawX = useMotionValue(0);
const rawY = useMotionValue(0);

// Spring physics — smooth, inertial camera rotation
const rotateY = useSpring(useTransform(rawX, [-0.5, 0.5], [-7, 7]),
{ stiffness: 70, damping: 18, mass: 0.4 });
const rotateX = useSpring(useTransform(rawY, [-0.5, 0.5], [ 5, -5]),
{ stiffness: 70, damping: 18, mass: 0.4 });

function onMouseMove(e) {
const r = e.currentTarget.getBoundingClientRect();
rawX.set((e.clientX - r.left) / r.width - 0.5); // Normalised [-0.5, 0.5]
rawY.set((e.clientY - r.top) / r.height - 0.5);
}

return (
<div onMouseMove={onMouseMove} onMouseLeave={() => { rawX.set(0); rawY.set(0); }}>
<motion.div style={{ rotateX, rotateY, transformStyle: "preserve-3d" }}>
{children}
</motion.div>
</div>
);
}

// FILE: components/ProblemHook.tsx — Sequential Problem Statement Display
// Cycles through the three-part problem statement at 1900ms intervals,
// then settles into the R&D label. This is the primary user hook.

const hooks = [
"A hospital has 200+ cooperation agreements.",
"Each interpreted differently by accounting.",
"One clause. Thousands in exposure.",
];

// FILE: components/SectionHow.tsx — Three-Step Method Visualisation
// Renders the patented three-step methodology (Agreement Ingestion →
// Domain Training → Live Compliance) with scroll-triggered animation.

// FILE: components/SectionContact.tsx — Research Programme Enrollment
// Collects hospital finance contact details for the on-site immersion
// research programme. Submitted contacts become candidates for the
// on-site data collection phase of the Domain Training module.
```

5.6 Technology Stack & Dependencies

The following listing documents the full technology stack and dependency versions constituting the system implementation:

Listing 5.6 — package.json & tsconfig.json (production)

```
// TECHNOLOGY STACK & DEPENDENCIES
// FILE: package.json (excerpt)

{
"name": "healthledger-ai",
"dependencies": {
// AI / LLM layer
"@anthropic-ai/sdk": "^0.x", // Anthropic Managed Agents API
// (claude-opus-4-8 model)
// Frontend framework
"next": "14.x", // Next.js 14 — React framework
"react": "^18",
"react-dom": "^18",

// Animation & UX
"framer-motion": "^11", // Spring physics, scroll animations

// Type safety
"typescript": "^5"
},
"scripts": {
"dev": "next dev",
"build": "next build",
"start": "next start"
}
}

// tsconfig.json (excerpt)
{
"compilerOptions": {
"target": "ES2017",
"lib": ["dom", "dom.iterable", "esnext"],
"module": "esnext",
"moduleResolution": "bundler",
"jsx": "preserve",
"strict": true,
"paths": { "@/*": ["./*"] }
}
}
```

6. CLAIMS (DRAFT — FOR ATTORNEY REVIEW)

The following claims are presented in preliminary form for discussion with patent counsel. Scope, dependency structure, and formal language will be finalised in coordination with the prosecuting attorney. Reference is made throughout to Listings 5.1–5.6 above.

Claim 1 — Independent System Claim

A computer-implemented artificial intelligence system for healthcare accounting compliance, the system comprising: (a) a document ingestion module configured to parse cooperation agreements of a healthcare institution, segment the parsed content into individual clauses, classify each clause according to accounting obligation type, generate a vector embedding for each clause, and store the embeddings in a searchable vector database indexed by clause type and agreement identifier; (b) a persistent AI agent entity created via a managed agent API, having a stable agent identifier and carrying institution-specific instructions comprising the institution's chart of accounts, internal naming conventions, and historical interpretive decisions; (c) a retrieval-augmented generation engine configured to receive an accounting compliance query, retrieve semantically similar clauses from the vector database by embedding the query and performing approximate nearest-neighbour search, assemble an augmented prompt incorporating the retrieved clauses, and submit the augmented prompt to a session created against the persistent agent entity; and (d) a compliance output module configured to receive the streamed response from the agent session and extract therefrom: a compliance determination, source clause identifiers, parent agreement references, and a confidence indicator.

Claim 2 — Independent Method Claim

A computer-implemented method for generating real-time accounting compliance guidance for a hospital institution, the method comprising: (a) ingesting the hospital's cooperation agreements by parsing, clause segmentation, obligation-type classification, vector embedding, and indexing in a vector database; (b) capturing institutional accounting knowledge through on-site immersion with the institution's finance team, comprising observation of accounting decisions, structured knowledge elicitation interviews, and recording of interpretive decisions with rationales; (c) creating a persistent AI agent entity configured with institution-specific instructions derived from the captured knowledge; (d) upon receipt of an accounting compliance query: embedding the query, retrieving the top-K most semantically similar agreement clauses, assembling a retrieval-augmented prompt, and generating a compliance determination via a session against the persistent agent entity; and (e) returning the compliance determination with explicit citations of the source clause identifier and parent agreement, enabling the determination to be audited and defended.

Claim 3 — Independent Methodology Claim

A method for developing a domain-specific artificial intelligence system for healthcare accounting, the method comprising: (a) conducting a sustained on-site engagement with the accounting department of a target healthcare institution; (b) during said engagement, observing accounting decisions involving cooperation agreement clauses and recording each decision as a structured interpretation record comprising: the agreement clause reference, the accounting decision taken, the rationale provided by the accountant, and the date of decision; (c) eliciting tacit interpretive conventions from accounting staff through structured interviews and formalising them as a naming convention mapping; (d) assembling a domain-specific instruction set for an AI agent from the recorded interpretation records, naming convention mappings, and the institution's chart of accounts; and (e) deploying the AI agent with said institution-specific instruction set, enabling the agent to generate compliance determinations in the specific accounting language of the institution.

Claim 4 — Dependent on Claim 1

The system of Claim 1, wherein the persistent AI agent entity is implemented using the Anthropic Managed Agents API, and wherein the agent is instantiated by calling the agents.create() method with a named agent identifier, a target large language model, and the institution-specific instruction set, the resulting agent identifier being stored durably for reuse across all subsequent compliance sessions.

Claim 5 — Dependent on Claim 1

The system of Claim 1, wherein the retrieval-augmented generation engine constructs the augmented prompt by formatting each retrieved clause as a bracketed citation comprising the agreement identifier and clause identifier, and wherein the agent is instructed to cite these identifiers explicitly in every compliance determination.

Claim 6 — Dependent on Claim 1

The system of Claim 1, wherein the compliance output module delivers agent responses via token-by-token streaming using server-sent events, enabling real-time display of the compliance determination as it is generated, with target first-token latency below 800 milliseconds.

Claim 7 — Dependent on Claim 1

The system of Claim 1, wherein the institution-specific instructions of the persistent agent entity are updated incrementally as new interpretive decisions are captured post-deployment, and wherein such updates take effect for all subsequent sessions without requiring retraining of the underlying language model.

Claim 8 — Dependent on Claim 2

The method of Claim 2, wherein the on-site immersion phase extends for a period of between four and twelve weeks, and wherein the validation of interim system outputs by accounting staff is performed iteratively during said period, with feedback incorporated into the agent instruction set prior to production deployment.

Claim 9 — Dependent on Claim 1

The system of Claim 1, further comprising a frontend web application implemented in Next.js 14 with React 18, said application comprising: a ProblemHook component that sequentially displays the three-part problem statement at configurable intervals; a SectionHow component that presents the three-step inventive methodology with scroll-triggered animation; and a SectionContact component that collects contact details from hospital finance professionals for enrolment in the on-site immersion research programme.

Claim 10 — Independent Use Claim

Use of the system of any of Claims 1, 4, 5, 6, 7, or 9 for reducing financial exposure arising from the misinterpretation of cooperation agreement clauses in hospital accounting operations, by providing clause-traceable, institution-specific compliance determinations at the point of accounting decision.

7. ABSTRACT

HealthLedger AI is an artificial intelligence system providing real-time accounting compliance intelligence for hospital finance departments. The system ingests and semantically indexes a hospital network's cooperation agreements using a vector embedding and retrieval-augmented generation pipeline, creates a persistent AI agent entity configured with institution-specific accounting knowledge captured through on-site immersion with the hospital's finance team, and deploys a clause-traceable compliance interface that delivers auditable accounting determinations at the point of decision. The inventive contribution spans: the cooperation agreement ingestion and semantic indexing pipeline; the institution-specific managed agent architecture; the retrieval-augmented compliance determination mechanism with source clause citation; and the on-site immersion methodology for capturing tacit institutional accounting knowledge as AI training material. The system addresses the material financial exposure arising from cooperation agreement clause misinterpretation in hospital accounting operations, for which no prior art domain-specific AI solution exists.

8. INVENTOR DECLARATION

I, the undersigned inventor, hereby declare that:

- ■ I am the sole original inventor of the subject matter described and claimed in this disclosure document.
- ■ All information provided herein, including the source code listings, system descriptions, and claim drafts, is true and accurate to the best of my knowledge and belief.
- ■ I have not previously made any public disclosure, offer for sale, or public use of this invention in any manner that would prejudice the filing of a patent application in any jurisdiction.
- ■ I acknowledge my duty to disclose to patent counsel any prior art, related publications, or third-party systems of which I become aware that may be material to the patentability of the claims.
- ■ This disclosure has been prepared in good faith for the purpose of initiating patent prosecution and is submitted under attorney-client privilege.

Full Name: Maria Polychroniadou

Date: June 26, 2026

Signature:

Witnessed by: _____

Date Witnessed: _____